

Debugging with GDB: Breakpoints, Registers, Assembly

GDB quickstart

```
gdb ./lab_bin
set disassembly-flavor intel
info registers
break main          # or break *0x401160 (address)
run
stepi / nexti       # instruction stepping
x/20i $rip          # disassemble from current pc
x/s $rax             # show string at register
```

Reading the check live

```
(gdb) break *main+45
(gdb) run
(gdb) x/s $rdi        # the string it's comparing
(gdb) x/3i $rip       # see the cmp/jne
(gdb) set $rax=0      # FLIP the comparison (lab!)
(gdb) continue
```

This is 'patching at runtime' – the attacker trick of skipping a conditional.

Registers of interest (x86-64)

RIP = current instruction pointer (you care about this in exploits)

RBP/RSP = stack frame (also critical for stack overflow)

RDI/RSI/RDX = first 3 args (SysV calling conv)

RAX = return value

Debugger for future exploits

You'll set breakpoints at `ret` / `call` to hijack or observe control flow

You'll 'patch' return addresses in your brain before actually writing an overflow

gdb + pwndbg (the RE workhorse)

```
# install pwndbg ~/.gdbinit:  
git clone https://github.com/pwndbg/pwndbg && cd pwndbg && ./setup.sh  
# then: nice stack/registers visualization + better disasm
```

When you get stuck

`info proc mappings` – stack/heap/libc addresses (needed for ret2libc later)

`bt` backtrace when in a crash

Compare with the Ghidra decompile to align memory addresses with source

GDB is the microscope of RE. The day you can flip a `jnz` and watch the app behave differently, you're no longer a beginner.

← [Disassembly: objdump, Ghidra...](#)

[Stack Exploitation: Overflow t...](#)